In [35]:
```
from IPython.core.display import HTML
HTML("""
<style>
.usecase-title, .usecase-duration, .usecase-section-header {
    padding-left: 15px;
    padding-bottom: 10px;
    padding-top: 10px;
    padding-right: 15px;
    background-color: #0f9295;
    color: #fff;
}

.usecase-title {
    font-size: 1.7em;
    font-weight: bold;
}

.usecase-authors, .usecase-level, .usecase-skill {
    padding-left: 15px;
    padding-bottom: 7px;
    padding-top: 7px;
    background-color: #baeaeb;
    font-size: 1.4em;
    color: #121212;
}

.usecase-level-skill  {
    display: flex;
}

.usecase-level, .usecase-skill {
    width: 50%;
}

.usecase-duration, .usecase-skill {
    text-align: right;
    padding-right: 15px;
    padding-bottom: 8px;
    font-size: 1.4em;
}

.usecase-section-header {
    font-weight: bold;
    font-size: 1.5em;
}

.usecase-subsection-header, .usecase-subsection-blurb {
    font-weight: bold;
    font-size: 1.2em;
    color: #121212;
}

.usecase-subsection-blurb {
    font-size: 1em;
```

```
    font-style: italic;
}
</style>
""")
```

Out[35]:

Weather Impact on Public Transport Usage

Authored by: Rukshan Dias

Duration: 90 mins

Level: Undergraduate / Junior

Pre-requisite Skills: Python, Pandas, Data Analysis, Data Cleaning, Data Visualisation, Basic Machine Learning

Scenario

As a data analyst, I want to examine whether weather conditions, especially rainfall, affect public transport activity, so that I can better understand patterns in transport usage and determine whether weather is a useful predictor of demand.

What this use case will teach you

At the end of this use case you will:

- load and combine multiple transport activity CSV files
- clean transport and weather datasets
- engineer time-based and weather-based features
- merge datasets using a common date field
- perform exploratory data analysis using summary statistics and visualisations
- compare transport usage on rainy and non-rainy days
- build and evaluate a baseline Linear Regression model

- interpret model metrics, coefficients, and correlations
- identify limitations and suggest improvements for future analysis

Background

This use case explores how weather conditions, particularly rainfall, influence public transport usage.

Public transport activity often changes depending on external conditions such as weather, time of day, events, and commuting behaviour. In this analysis, the focus is on understanding whether rainfall has a measurable relationship with transport activity.

The analysis uses two datasets:

- transport activity count data collected across multiple locations and times
- Bureau of Meteorology weather data containing daily rainfall values

The transport data from several monthly CSV files is combined into one dataset, while the weather data is prepared separately and then merged using a shared date field. After cleaning and feature engineering, the merged dataset is explored using visualisations and summary statistics.

The use case then builds a baseline Linear Regression model using rainfall and hour as input features to predict transport usage. The results show that rainfall has only a very weak relationship with transport activity, while hour of day has a stronger influence due to daily commuting patterns.

This use case is useful for understanding how to combine open datasets, perform exploratory analysis, and build an introductory predictive model in a real-world transport setting.

Load Datasets

We load the transport activity and weather datasets for analysis.

In [36]:
```
import requests
import zipfile
import io
import pandas as pd

api_url = "https://data.melbourne.vic.gov.au/api/explore/v2.1/catal

params = {
    "limit": 20,
    "timezone": "Australia/Melbourne"
}
```

```
api_response = requests.get(api_url, params=params)
api_response.raise_for_status()

api_data = api_response.json()

print("API v2.1 status code:", api_response.status_code)
print("API v2.1 total records:", api_data.get("total_count"))
print("API v2.1 records returned:", len(api_data.get("results", [])

traffic_url = "https://opendatasoft-s3.s3.ap-southeast-2.amazonaws.

response = requests.get(traffic_url)
response.raise_for_status()

zip_file = zipfile.ZipFile(io.BytesIO(response.content))

traffic_dfs = []

for file_name in zip_file.namelist():
    if file_name.endswith(".csv"):
        with zip_file.open(file_name) as f:
            df = pd.read_csv(f)
            traffic_dfs.append(df)

traffic = pd.concat(traffic_dfs, ignore_index=True)

print("Traffic shape:", traffic.shape)
print(traffic.head())
```

```
API v2.1 status code: 200
API v2.1 total records: 0
API v2.1 records returned: 0
Traffic shape: (5421538, 8)
   countLocationId               countLocationName  CountLocationLat
\
0            45040  Queens Bridge Street - CoM1549        -37.820920
1            46417  Princes Park Asset ID: COM0706        -37.779953
2            45474   La Trobe St- William St I-Hub        -37.811920
3            51281     Yarra Promenade - West Path        -37.822354
4            45473   La Trobe St- William St I-Hub        -37.811920

   CountLocationLong                      from
to  \
0         144.961642  2025-04-01T00:00:00.000Z  2025-04-01T00:05:00.
000Z
1         144.962578  2025-04-01T00:00:00.000Z  2025-04-01T00:05:00.
000Z
2         144.956251  2025-04-01T00:00:00.000Z  2025-04-01T00:05:00.
000Z
3         144.958747  2025-04-01T00:00:00.000Z  2025-04-01T00:05:00.
000Z
4         144.956251  2025-04-01T00:00:00.000Z  2025-04-01T00:05:00.
000Z

        class  count
0     cyclist      1
1     cyclist      7
2  pedestrian      8
3  pedestrian     12
4   motorbike      1
```

In [40]:

```python
import requests
import zipfile
import io
import os
import pandas as pd

weather_url = "https://www.bom.gov.au/tmp/cdio/IDCJAC0009_086338_20
local_weather_path = "DEPENDENCIES/IDCJAC0009_086338_2025.csv"

headers = {"User-Agent": "Mozilla/5.0"}

try:
    response = requests.get(weather_url, headers=headers, timeout=3
    response.raise_for_status()

    zip_file = zipfile.ZipFile(io.BytesIO(response.content))
    csv_file = [f for f in zip_file.namelist() if f.endswith(".csv"

    with zip_file.open(csv_file) as f:
        weather = pd.read_csv(f, skiprows=10, header=None)

    print("Weather data loaded from BOM ZIP URL.")

except Exception as e:
```

```
    print("BOM temporary URL failed.")
    print("Loading weather data from local DEPENDENCIES folder.")
    print("Reason:", e)

    weather = pd.read_csv(local_weather_path, skiprows=10, header=N

weather.columns = [
    "product_code",
    "station_number",
    "Year",
    "Month",
    "Day",
    "rainfall",
    "quality",
    "flag"
]

print("Weather shape:", weather.shape)
print(weather.head())
```

```
Weather data loaded from BOM ZIP URL.
Weather shape: (356, 8)
  product_code  station_number  Year  Month  Day  rainfall  quality
flag
0   IDCJAC0009           86338  2025      1   10       0.0      1.0
Y
1   IDCJAC0009           86338  2025      1   11       0.0      1.0
Y
2   IDCJAC0009           86338  2025      1   12       0.0      1.0
Y
3   IDCJAC0009           86338  2025      1   13      31.0      1.0
Y
4   IDCJAC0009           86338  2025      1   14       0.0      1.0
Y
```

Data Cleaning

The datasets are cleaned by handling missing values, removing duplicates, and ensuring consistent formatting.

In [41]:
```
traffic.drop_duplicates(inplace=True)
weather.drop_duplicates(inplace=True)

traffic = traffic.ffill()
weather = weather.ffill()

print("Traffic missing values:")
print(traffic.isnull().sum())

print("\nWeather missing values:")
print(weather.isnull().sum())
```

```
Traffic missing values:
countLocationId      0
countLocationName    0
CountLocationLat     0
CountLocationLong    0
from                 0
to                   0
class                0
count                0
dtype: int64

Weather missing values:
product_code      0
station_number    0
Year              0
Month             0
Day               0
rainfall          0
quality           0
flag              0
dtype: int64
```

Feature Engineering

Create time-based and weather-based features.

In [42]:
```
# Convert traffic time
traffic['from'] = pd.to_datetime(traffic['from'])

# Extract hour
traffic['hour'] = traffic['from'].dt.hour

# Create date column
traffic['date'] = traffic['from'].dt.date
traffic['date'] = pd.to_datetime(traffic['date'])
```

In [43]:
```
weather['date'] = pd.to_datetime(weather[['Year', 'Month', 'Day']])

weather = weather.rename(columns={
    'Rainfall amount (millimetres)': 'rainfall'
})
```

Merge Data

Combine transport and weather datasets using date.

In [44]:
```
data = pd.merge(
    traffic,
    weather[['date', 'rainfall']],
    on='date',
    how='left'
)

print("Merged shape:", data.shape)
```

```
print(data.head())
print("\nMissing values after merge:")
print(data.isnull().sum())
```

```
Merged shape: (4721358, 11)
   countLocationId              countLocationName  CountLocationLat
\
0            45040  Queens Bridge Street - CoM1549        -37.820920
1            46417  Princes Park Asset ID: COM0706        -37.779953
2            45474   La Trobe St- William St I-Hub        -37.811920
3            51281     Yarra Promenade - West Path        -37.822354
4            45473   La Trobe St- William St I-Hub        -37.811920

   CountLocationLong                      from
to  \
0         144.961642 2025-04-01 00:00:00+00:00  2025-04-01T00:05:00.
000Z
1         144.962578 2025-04-01 00:00:00+00:00  2025-04-01T00:05:00.
000Z
2         144.956251 2025-04-01 00:00:00+00:00  2025-04-01T00:05:00.
000Z
3         144.958747 2025-04-01 00:00:00+00:00  2025-04-01T00:05:00.
000Z
4         144.956251 2025-04-01 00:00:00+00:00  2025-04-01T00:05:00.
000Z

        class  count  hour       date  rainfall
0     cyclist      1     0 2025-04-01       0.0
1     cyclist      7     0 2025-04-01       0.0
2  pedestrian      8     0 2025-04-01       0.0
3  pedestrian     12     0 2025-04-01       0.0
4   motorbike      1     0 2025-04-01       0.0

Missing values after merge:
countLocationId           0
countLocationName         0
CountLocationLat          0
CountLocationLong         0
from                      0
to                        0
class                     0
count                     0
hour                      0
date                      0
rainfall             109008
dtype: int64
```

Exploratory Data Analysis

In this section, we explore the relationship between rainfall and public transport usage. The aim is to identify patterns, trends, and possible correlations before building a prediction model.

In [45]:
```
print("Columns in merged dataset:")
print(data.columns)

print("\nBasic info:")
print(data.info())

print("\nSummary statistics:")
print(data.describe(include='all'))
```

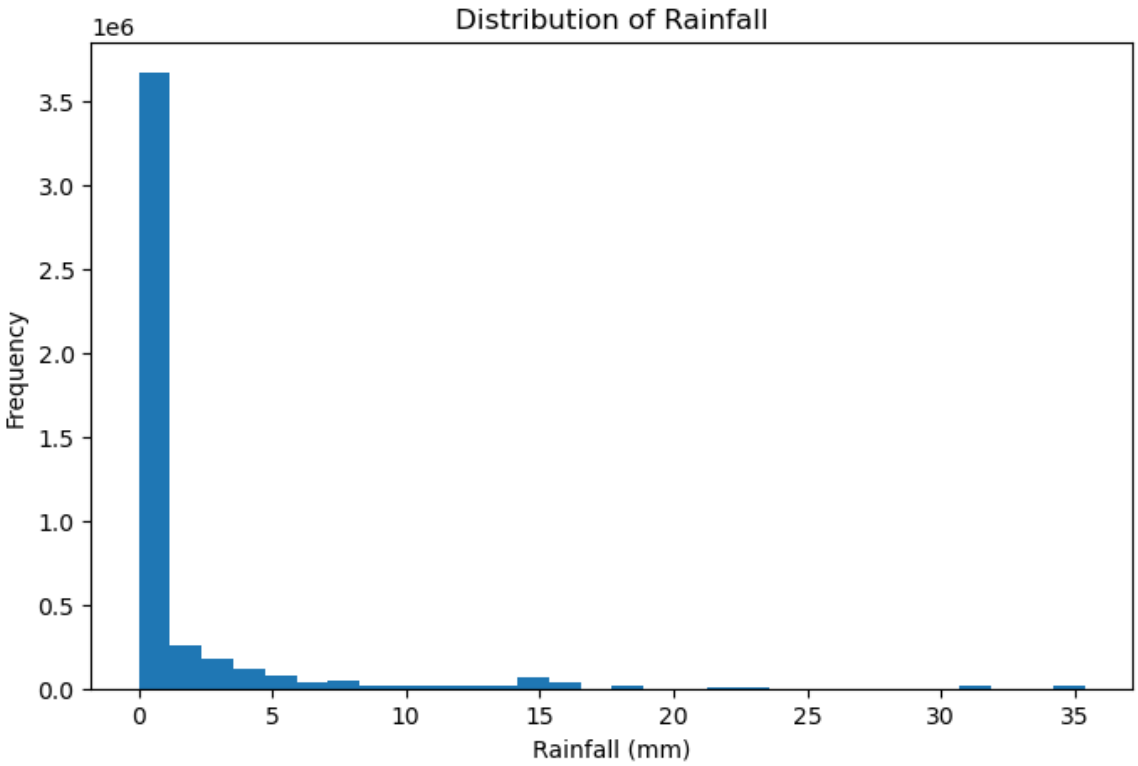
```
Columns in merged dataset:
Index(['countLocationId', 'countLocationName', 'CountLocationLat',
       'CountLocationLong', 'from', 'to', 'class', 'count', 'hour',
'date',
       'rainfall'],
      dtype='object')

Basic info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4721358 entries, 0 to 4721357
Data columns (total 11 columns):
 #   Column             Dtype
---  ------             -----
 0   countLocationId    int64
 1   countLocationName  object
 2   CountLocationLat   float64
 3   CountLocationLong  float64
 4   from               datetime64[ns, UTC]
 5   to                 object
 6   class              object
 7   count              int64
 8   hour               int32
 9   date               datetime64[ns]
 10  rainfall           float64
dtypes: datetime64[ns, UTC](1), datetime64[ns](1), float64(3), int3
2(1), int64(2), object(3)
memory usage: 378.2+ MB
None

Summary statistics:
        countLocationId             countLocationName  CountLocatio
nLat  \
count      4.721358e+06                       4721358      4.721358
e+06
unique              NaN                            23
NaN
top                 NaN  La Trobe St- William St I-Hub
NaN
freq                NaN                       1376018
NaN
mean       4.638385e+04                           NaN     -3.780908
e+01
min        4.377200e+04                           NaN     -3.782392
e+01
25%        4.504000e+04                           NaN     -3.782018
e+01
50%        4.547200e+04                           NaN     -3.781192
```

```
e+01
75%        4.641700e+04                              NaN     -3.780816
e+01
max        5.809700e+04                              NaN     -3.777977
e+01
std        2.677084e+03                              NaN      1.338081
e-02

        CountLocationLong                                from  \
count        4.721358e+06                             4721358
unique                NaN                                 NaN
top                   NaN                                 NaN
freq                  NaN                                 NaN
mean         1.449598e+02  2025-07-05 12:28:15.970356736+00:00
min          1.449415e+02            2025-01-01 00:00:00+00:00
25%          1.449563e+02            2025-04-05 02:10:00+00:00
50%          1.449603e+02            2025-07-06 19:15:00+00:00
75%          1.449626e+02            2025-10-05 23:05:00+00:00
max          1.449876e+02            2025-12-31 23:55:00+00:00
std          6.495471e-03                                 NaN

                            to       class         count
hour  \
count                   4721358     4721358  4.721358e+06  4.721358
e+06
unique                   105018          14           NaN
NaN
top     2025-12-11T09:00:00.000Z  pedestrian           NaN
NaN
freq                         86     1861620           NaN
NaN
mean                        NaN         NaN  1.345293e+01  1.271867
e+01
min                         NaN         NaN  1.000000e+00  0.000000
e+00
25%                         NaN         NaN  1.000000e+00  8.000000
e+00
50%                         NaN         NaN  3.000000e+00  1.300000
e+01
75%                         NaN         NaN  1.100000e+01  1.800000
e+01
max                         NaN         NaN  1.904000e+03  2.300000
e+01
std                         NaN         NaN  3.280250e+01  6.033426
e+00

                               date      rainfall
count                       4721358  4.612350e+06
unique                          NaN           NaN
top                             NaN           NaN
freq                            NaN           NaN
mean    2025-07-04 23:17:35.977369856  1.413405e+00
min               2025-01-01 00:00:00  0.000000e+00
25%               2025-04-05 00:00:00  0.000000e+00
50%               2025-07-06 00:00:00  0.000000e+00
75%               2025-10-05 00:00:00  6.000000e-01
```

```
max              2025-12-31 00:00:00  3.540000e+01
std                              NaN  4.051286e+00
```

Rainfall Distribution

In [46]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))
plt.hist(data['rainfall'].dropna(), bins=30)
plt.xlabel('Rainfall (mm)')
plt.ylabel('Frequency')
plt.title('Distribution of Rainfall')
plt.show()
```



This histogram shows the distribution of rainfall values across the dataset. The majority of observations are heavily concentrated near zero, indicating that most days experience little to no rainfall.

There is a long right tail in the distribution, representing a small number of days with higher rainfall levels. This suggests that heavy rainfall events are relatively rare compared to dry or low-rainfall days.

Overall, the distribution is highly skewed, which is typical for weather data and indicates that rainfall is not evenly distributed across time.

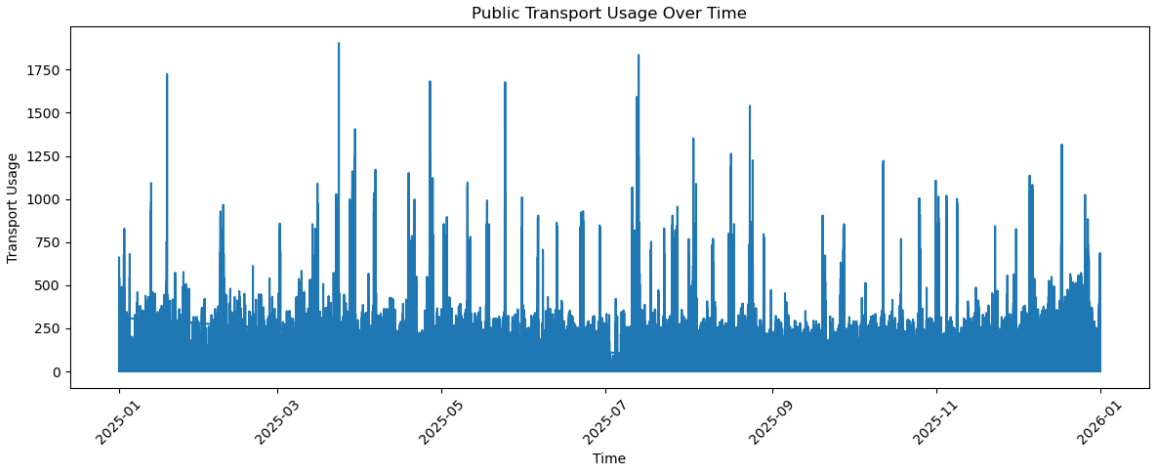
Public Transport Usage Over Time

This time series plot shows how public transport usage changes over time across the dataset.

In [47]:
```
print(data.columns)
```

```
Index(['countLocationId', 'countLocationName', 'CountLocationLat',
       'CountLocationLong', 'from', 'to', 'class', 'count', 'hour',
'date',
       'rainfall'],
      dtype='object')
```

In [48]:
```
plt.figure(figsize=(12, 5))
plt.plot(data['from'], data['count'])
plt.xlabel('Time')
plt.ylabel('Transport Usage')
plt.title('Public Transport Usage Over Time')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



The graph displays significant fluctuations in transport usage, with frequent spikes and drops throughout the year. These variations indicate that transport demand is not constant and changes depending on time and external factors.

Although the data is dense and individual patterns are difficult to isolate, the overall trend suggests recurring peaks which are likely associated with daily commuting behaviour and high-demand periods.

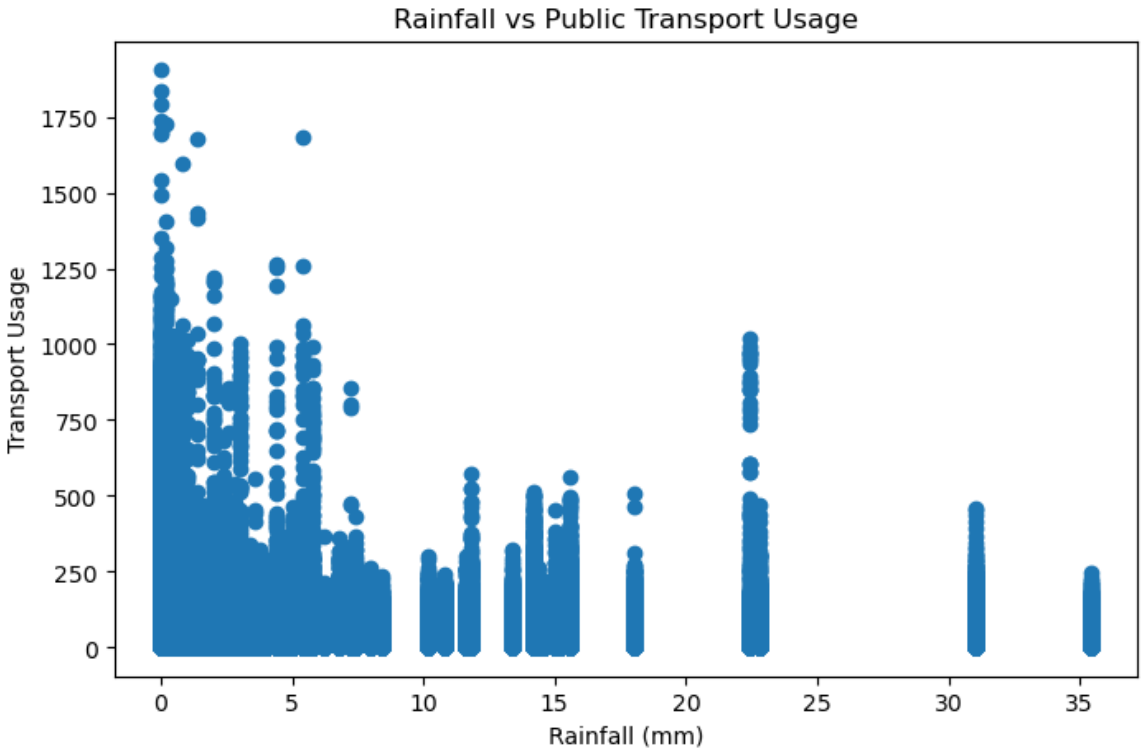
This reinforces the idea that time-related factors play a key role in influencing transport usage patterns.

Rainfall vs Transport Usage

This scatter plot illustrates the relationship between rainfall levels and public transport usage.

In [49]:
```
plt.figure(figsize=(8, 5))
plt.scatter(data['rainfall'], data['count'])
plt.xlabel('Rainfall (mm)')
plt.ylabel('Transport Usage')
plt.title('Rainfall vs Public Transport Usage')
```

```
plt.show()
```



The data points are widely scattered, showing no clear linear pattern between rainfall and transport activity. This indicates that rainfall alone is not a strong predictor of transport usage.

While there is a slight tendency for transport usage to decrease at higher rainfall levels, the variation remains large, and the relationship is weak and inconsistent.

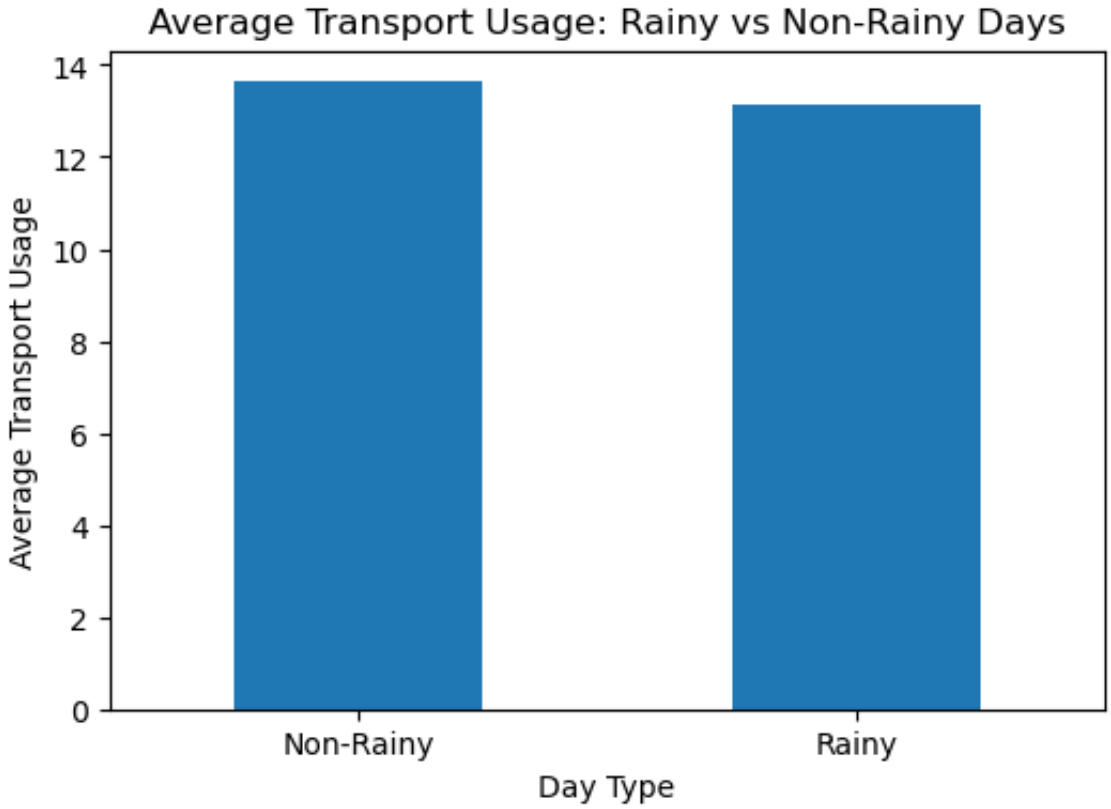
Overall, the plot suggests that other factors, such as time of day or location, have a much greater influence on transport demand.

Average Transport Usage on Rainy and Non-Rainy Days

To simplify interpretation, rainfall values are converted into two categories: rainy and non-rainy days. This allows a direct comparison of average transport usage under each condition.

In [50]:
```
data['rainy_day'] = data['rainfall'].apply(lambda x: 1 if x > 0 els

avg_usage = data.groupby('rainy_day')['count'].mean()
print(avg_usage)

plt.figure(figsize=(6, 4))
avg_usage.plot(kind='bar')
plt.xticks([0, 1], ['Non-Rainy', 'Rainy'], rotation=0)
plt.xlabel('Day Type')
plt.ylabel('Average Transport Usage')
plt.title('Average Transport Usage: Rainy vs Non-Rainy Days')
plt.show()
```

```
rainy_day
0    13.630065
1    13.124919
Name: count, dtype: float64
```



The bar chart shows that the average transport usage is very similar for both rainy and non-rainy days. The difference between the two values is minimal.

This indicates that rainfall does not have a significant impact on overall transport usage in this dataset.

The results suggest that other factors, such as time of day and daily commuting patterns, play a more important role in influencing transport demand.

Model Building

A Linear Regression model is used as a baseline to predict transport usage based on rainfall and time of day.

This helps evaluate whether a simple linear relationship exists between these variables and transport activity.

In [51]:
```
TARGET = 'count'
```

In [52]:
```
model_data = data[['rainfall', 'hour', TARGET]].copy()
```

```
model_data = model_data.dropna()

print("Model dataset shape:", model_data.shape)
model_data.head()
```

```
Model dataset shape: (4612350, 3)
```

Out[52]:

	rainfall	hour	count
0	0.0	0	1
1	0.0	0	7
2	0.0	0	8
3	0.0	0	12
4	0.0	0	1

Train-Test Split

The dataset is divided into training and testing sets. The training set is used to train the model, while the testing set is used to evaluate its predictive performance.

In [53]:
```
from sklearn.model_selection import train_test_split

X = model_data[['rainfall', 'hour']]
y = model_data[TARGET]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)
```

```
Training set size: (3689880, 2)
Testing set size: (922470, 2)
```

Linear Regression Model

A Linear Regression model is used as a baseline model to predict transport usage.

This model is suitable for understanding whether there is a simple linear relationship between rainfall, hour, and transport activity.

In [54]:
```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

print("Model training completed.")
```

```
Model training completed.
```

Model Evaluation

Evaluate model performance.

In [55]:
```
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

# Predictions
y_pred = model.predict(X_test)

# Metrics
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R²:", r2)
```

```
MAE: 15.838178016481782
MSE: 1061.3655060804901
RMSE: 32.5786050358282
R²: 0.009484412266122066
```

Model Performance Interpretation

The model shows a very low R-squared (R^2) value, indicating that it explains only a small portion of the variance in transport usage.

This suggests that rainfall and hour alone are not sufficient predictors of transport activity. While the model provides a basic baseline, it does not capture the complexity of real-world transport patterns.

Additional features such as location, day of the week, or special events would likely improve model performance.

Fine-Tuning

In this stage, the model is refined and evaluated to better understand its performance and limitations. The dataset was prepared by selecting relevant features, specifically rainfall and hour, which were expected to influence transport usage.

Missing values were removed to ensure clean input data for the model, and a Linear Regression model was trained using the processed dataset. The model was then evaluated using multiple performance metrics including MAE, MSE, RMSE, and R^2.

Further validation was performed using correlation analysis, coefficient interpretation, and visualisations. These steps confirm that while the model is technically correct, it has limited predictive power due to weak feature relationships.

In [56]:
```
# Fine-tuning summary
print("Features used:", list(X.columns))
print("Model coefficients:", model.coef_)
print("R² Score:", r2)
```

```
Features used: ['rainfall', 'hour']
Model coefficients: [-0.06682587  0.52626836]
R² Score: 0.009484412266122066
```

Fine-Tuning Improvement: Alternative Model

To further refine the analysis, a Random Forest Regressor was tested as an alternative model. Unlike Linear Regression, this model can capture non-linear relationships in the data.

However, the performance improvement was limited, indicating that the main limitation lies in the lack of strong predictive features rather than the model choice.

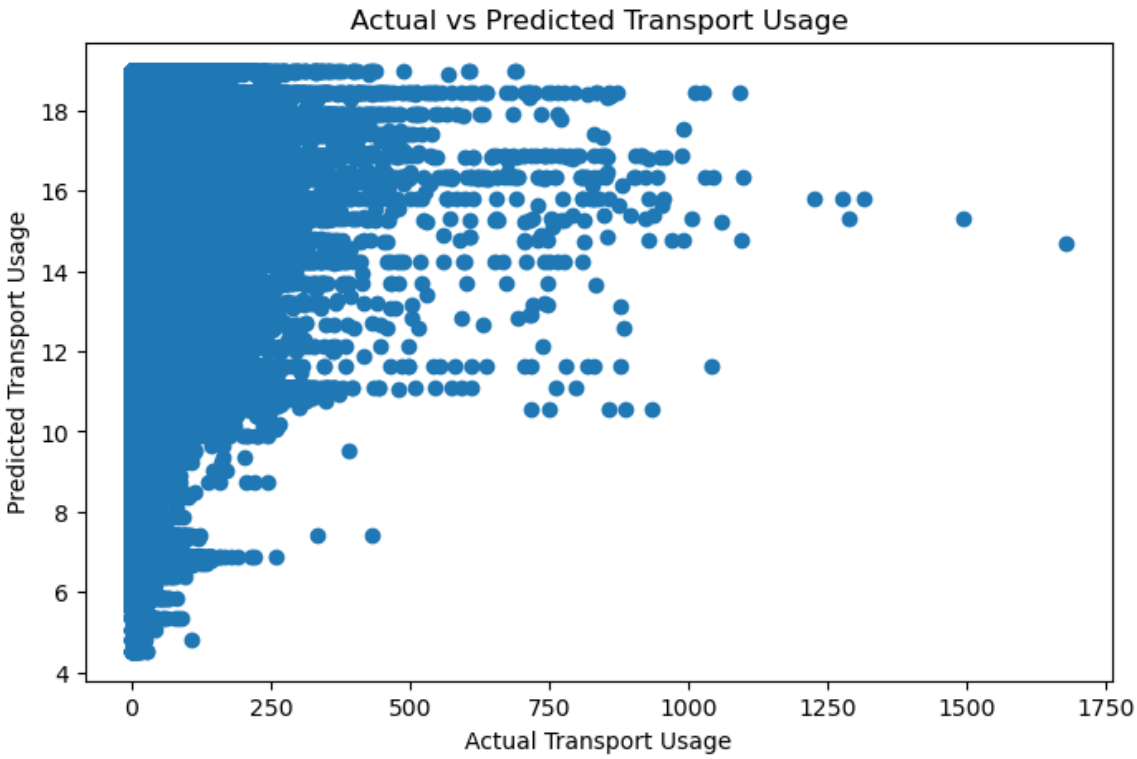
In [57]:
```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score

rf_model = RandomForestRegressor(n_estimators=50, random_state=42)
rf_model.fit(X_train, y_train)

rf_pred = rf_model.predict(X_test)

print("Random Forest R²:", r2_score(y_test, rf_pred))
```

```
Random Forest R²: 0.019419170102387695
```

Actual vs Predicted Values

This scatter plot compares the actual transport usage values with the values predicted by the Linear Regression model.

In [58]:
```
plt.figure(figsize=(8, 5))
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Transport Usage")
plt.ylabel("Predicted Transport Usage")
plt.title("Actual vs Predicted Transport Usage")
plt.show()
```



The points show a weak alignment and do not follow a clear diagonal pattern, which would be expected if the model predictions were accurate.

Instead, the predictions are widely dispersed across the range of actual values, indicating that the model is unable to reliably capture the underlying relationship in the data.

This suggests that the model has low predictive performance and that the current features, such as rainfall and hour, are insufficient to explain variations in transport usage.

Overall, the results confirm that additional variables and more complex modelling approaches would be required to improve prediction accuracy.

The rainfall coefficient is slightly negative, suggesting that as rainfall increases, transport usage tends to decrease. However, the value is very close to zero, indicating that the effect is minimal and not practically significant.

In contrast, the coefficient for hour is positive and substantially larger than rainfall, indicating that transport usage increases as the day progresses. This reflects typical commuting patterns, where activity peaks during working hours.

Overall, the coefficients confirm that time of day is a more important predictor of transport usage than rainfall in this model.

In [59]:
```
coefficients = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_
})

print(coefficients)
```

```
   Feature  Coefficient
0  rainfall    -0.066826
1      hour     0.526268
```

Key Findings

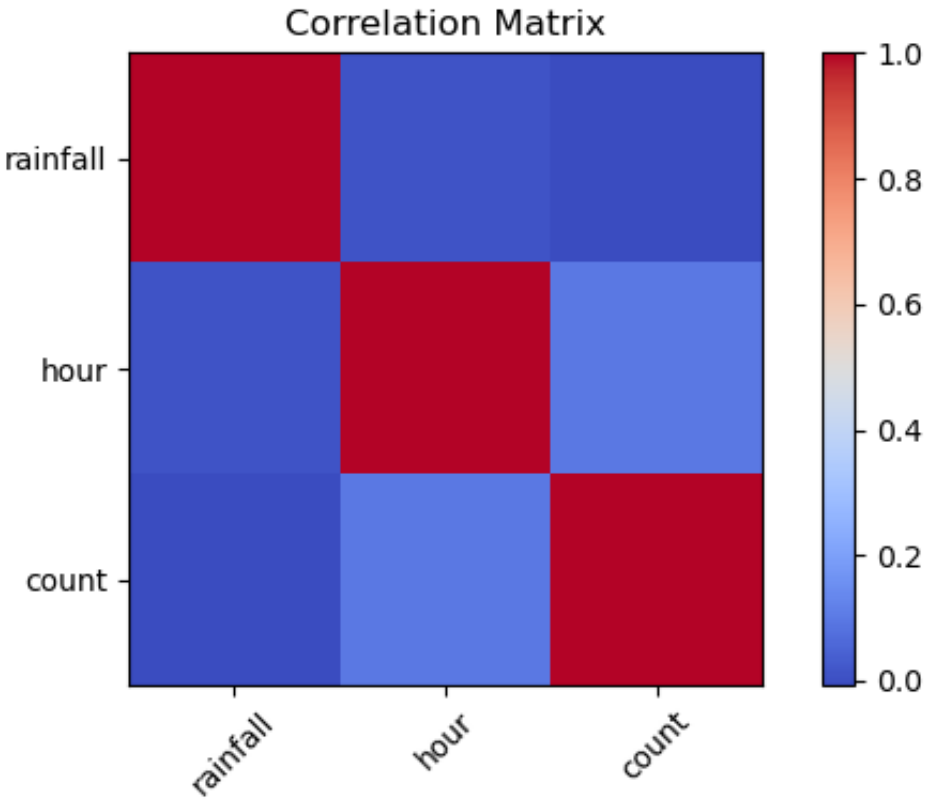
- Rainfall has a very weak and negligible impact on transport usage
- Transport activity remains similar across rainy and non-rainy conditions
- Time of day (hour) has a stronger and more meaningful influence
- The model performance is low, indicating limited predictive capability

Correlation Analysis

A correlation matrix is used to examine the linear relationship between rainfall, hour, and transport usage.

In [60]:
```
corr_data = model_data[['rainfall', 'hour', 'count']].corr()
print(corr_data)

import matplotlib.pyplot as plt

plt.figure(figsize=(6, 4))
plt.imshow(corr_data, cmap='coolwarm', interpolation='nearest')
plt.colorbar()
plt.xticks(range(len(corr_data.columns)), corr_data.columns, rotati
plt.yticks(range(len(corr_data.columns)), corr_data.columns)
plt.title('Correlation Matrix')
plt.tight_layout()
plt.show()
```

```
          rainfall      hour     count
rainfall  1.000000  0.008676 -0.007482
hour      0.008676  1.000000  0.096636
count    -0.007482  0.096636  1.000000
```



The correlation between rainfall and transport usage is extremely close to zero (approximately -0.007), indicating no meaningful linear relationship between rainfall and transport activity.

In contrast, the correlation between hour and transport usage is slightly positive (approximately 0.096), suggesting that time of day has a small but noticeable influence on transport activity.

The correlation heatmap visually confirms these findings, with rainfall showing near-zero association and hour showing a slightly stronger relationship with transport usage.

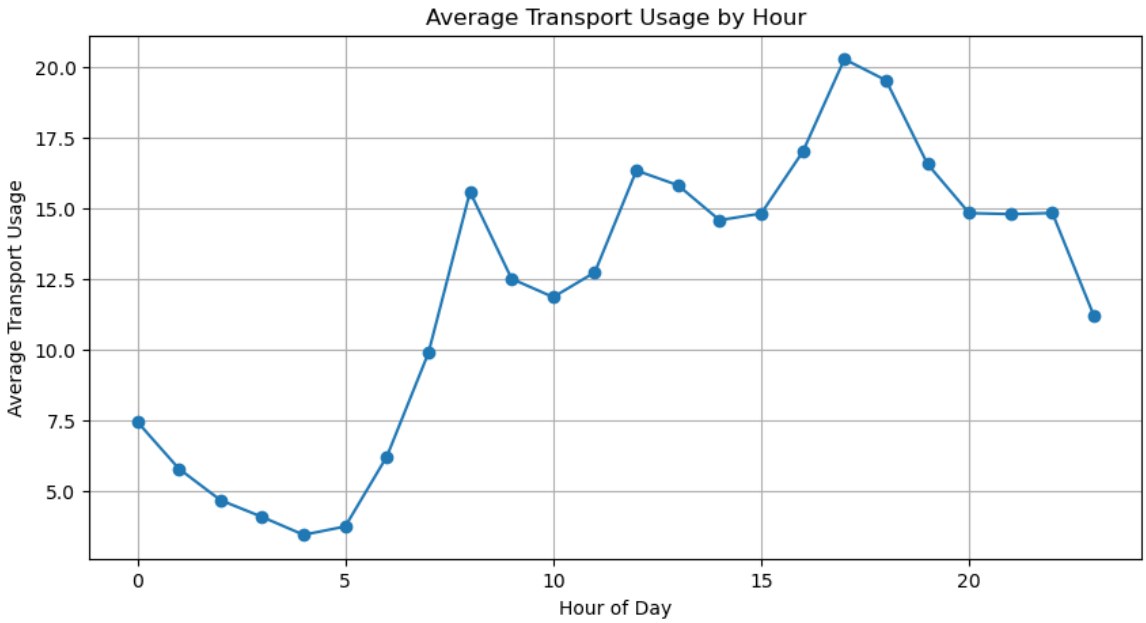
Overall, the results indicate that rainfall is not a strong predictor of transport usage, while time-related factors play a more important role.

Hourly Transport Usage Pattern

This graph shows how average transport usage varies across different hours of the day, revealing clear daily patterns in transport activity.

In [61]:
```
hourly_usage = data.groupby('hour')['count'].mean()

plt.figure(figsize=(10, 5))
plt.plot(hourly_usage.index, hourly_usage.values, marker='o')
plt.xlabel('Hour of Day')
plt.ylabel('Average Transport Usage')
plt.title('Average Transport Usage by Hour')
plt.grid(True)
```

```
plt.show()
```



Transport usage is lowest during early morning hours (approximately 2 AM to 5 AM), reflecting minimal travel activity during these times.

Usage increases significantly from around 6 AM onwards, corresponding to the morning commute period. A steady rise continues through the late morning and early afternoon.

The highest transport usage occurs during the late afternoon (around 4 PM to 6 PM), which aligns with typical evening commuting hours when people return home from work or study.

After this peak, transport usage gradually declines into the evening and night.

These patterns clearly demonstrate that time of day has a strong influence on transport usage, significantly more than rainfall, as activity is driven by consistent human commuting behaviour.

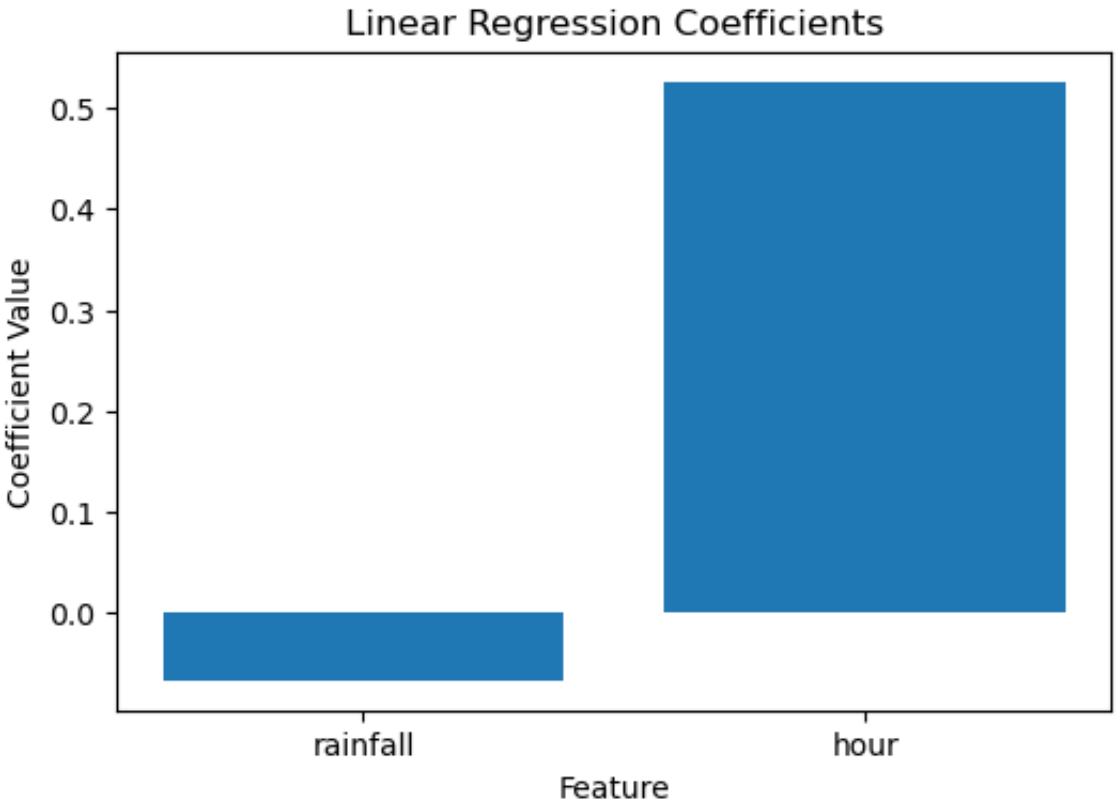
Interpretation of Model Coefficients

The Linear Regression model assigns coefficients to each feature, indicating their influence on predicted transport usage.

In [62]:
```
coefficients = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_
})

print(coefficients)

plt.figure(figsize=(6, 4))
plt.bar(coefficients['Feature'], coefficients['Coefficient'])
```

```
plt.xlabel('Feature')
plt.ylabel('Coefficient Value')
plt.title('Linear Regression Coefficients')
plt.show()
```

```
    Feature  Coefficient
0  rainfall    -0.066826
1      hour     0.526268
```



The coefficient for rainfall is slightly negative (approximately -0.062), suggesting that increased rainfall is associated with a small decrease in transport usage. However, the value is very close to zero, indicating that the effect is negligible and not practically significant.

In contrast, the coefficient for hour is positive and substantially larger (approximately 0.523), indicating that transport usage increases as the day progresses. This reflects typical daily commuting patterns where activity is higher during working hours.

The difference in magnitude between the coefficients shows that time of day has a much stronger influence on transport usage compared to rainfall.

Overall, the model confirms that temporal patterns are the primary driver of transport activity, while weather conditions such as rainfall have minimal impact.

Interpretation of Correlation Matrix

The correlation matrix shows the relationship between rainfall, hour, and transport usage (count).

Rainfall has a correlation value close to zero with transport usage, indicating that there is almost no linear relationship between rainfall and transport activity in this dataset.

Hour shows a slightly positive correlation with transport usage, suggesting that time of day has a small influence on activity levels.

Overall, the correlation results indicate that rainfall is not a strong predictor of transport usage, while hour has a slightly more noticeable but still weak effect.

Interpretation of Hourly Transport Usage

The hourly transport usage graph shows clear patterns throughout the day.

Transport usage is lowest during early morning hours (around 2 AM to 5 AM), increases sharply in the morning, and reaches higher levels during the afternoon and evening.

Peak usage appears in the late afternoon (around 4 PM to 6 PM), which likely corresponds to commuting periods.

This confirms that time of day is a much more significant factor influencing transport usage compared to rainfall.

Interpretation of Model Coefficients

The Linear Regression model provides coefficients that indicate how each feature affects transport usage.

The coefficient for rainfall is slightly negative, meaning that as rainfall increases, transport usage tends to decrease. However, the value is very close to zero, indicating that the effect is minimal.

The coefficient for hour is positive and significantly larger than rainfall, showing that transport usage increases as the day progresses.

This confirms that hour is a more important feature than rainfall in predicting transport activity in this model.

Final Insights

The analysis shows that rainfall has almost no measurable impact on

transport usage in this dataset.

Both the correlation analysis and regression model indicate that rainfall is not a strong predictor of activity levels.

On the other hand, time of day (hour) plays a much more important role, with clear patterns showing increased usage during typical commuting hours.

The Linear Regression model was successfully trained and evaluated, but the low correlation values suggest that the current features are not sufficient to build an accurate predictive model.

Overall, transport usage appears to be driven more by daily human behaviour patterns rather than weather conditions alone.

Conclusion

The analysis shows that rainfall has minimal impact on transport usage, while time of day plays a more significant role.

The Linear Regression model confirms that the current features are insufficient for accurate prediction, and additional variables would be needed to improve model performance.

Overall, transport activity appears to be driven more by daily human behaviour patterns than by weather conditions alone.

LLM-Based Traffic Explanation Module

To enhance interpretability, this section implements an AI-style explanation module that allows users to ask natural-language questions about transport conditions and receive human-readable explanations.

Instead of only displaying model outputs, the system combines predictive analytics with rule-based natural-language reasoning to explain why congestion or high transport usage may occur.

In [63]:
```
import re
import pandas as pd

def extract_query_features(query):
    query_lower = query.lower()

    hour = None
```

```
    match_12hr = re.search(r'(\d{1,2})\s*(am|pm)', query_lower)

    if match_12hr:
        hour = int(match_12hr.group(1))
        period = match_12hr.group(2)

        if period == "pm" and hour != 12:
            hour += 12
        elif period == "am" and hour == 12:
            hour = 0
    else:
        match_24hr = re.search(r'\b([01]?\d|2[0-3])\b', query_lower
        if match_24hr:
            hour = int(match_24hr.group(1))

    rainfall = 0.0
    if "rain" in query_lower or "raining" in query_lower or "wet" i
        rainfall = data['rainfall'].mean()

    location = "Melbourne CBD" if "melbourne" in query_lower or "cb

    return hour, rainfall, location


def traffic_explanation_assistant(query):
    hour, rainfall, location = extract_query_features(query)

    if hour is None:
        hour = int(data['hour'].mean())

    input_data = pd.DataFrame({
        "rainfall": [rainfall],
        "hour": [hour]
    })

    predicted_usage = model.predict(input_data)[0]

    if hour in [7, 8, 9, 16, 17, 18]:
        time_reason = "This is a peak commuting period, so transpor
    elif hour < 6:
        time_reason = "This is early morning, so transport usage is
    else:
        time_reason = "This is outside the strongest peak period, s

    if rainfall > 0:
        weather_reason = "Rainfall was detected, but the analysis s
    else:
        weather_reason = "No rainfall condition was detected."

    if predicted_usage >= data['count'].mean():
        level = "higher than average"
    else:
        level = "lower than average"

    response = f"""
AI Traffic Explanation
```

```
User question:
{query}

Prediction:
The predicted transport usage near {location} at {hour}:00 is {leve

Explanation:
{time_reason}
{weather_reason}

Recommendation:
Allow extra travel time during peak hours, especially if the weathe
"""

    return response


user_question = input("Ask a traffic/weather question: ")

print(traffic_explanation_assistant(user_question))
```

```
AI Traffic Explanation

User question:
Will transport usage be busy near Melbourne CBD at 5PM if it is rain
ing?

Prediction:
The predicted transport usage near Melbourne CBD at 17:00 is higher
than average.

Explanation:
This is a peak commuting period, so transport usage is likely to be
higher.
Rainfall was detected, but the analysis showed rainfall has only a w
eak effect on usage.

Recommendation:
Allow extra travel time during peak hours, especially if the weather
is wet.
```

Prompt Engineering and RAG-Based Traffic Explanation Module

This section integrates prompt engineering and Retrieval-Augmented Generation (RAG) into the transport analysis workflow. The system retrieves relevant analytical results from the dataset and model outputs, then uses a structured prompt to generate a human-readable traffic explanation.

In [64]:

```
import re
import pandas as pd

def retrieve_rag_context(user_query):
    rainfall_corr = data[['rainfall', 'count']].corr().iloc[0, 1]
    avg_usage = data['count'].mean()
    avg_rainfall = data['rainfall'].mean()
    rainy_avg = data.groupby('rainy_day')['count'].mean().loc[1]
    non_rainy_avg = data.groupby('rainy_day')['count'].mean().loc[0

    context = f"""
Retrieved analytical context:

Average transport usage: {avg_usage:.2f}
Average rainfall: {avg_rainfall:.2f} mm
Rainfall vs transport usage correlation: {rainfall_corr:.3f}
Average usage on rainy days: {rainy_avg:.2f}
Average usage on non-rainy days: {non_rainy_avg:.2f}
Linear Regression R2 score: {r2:.3f}
Model coefficients: rainfall = {model.coef_[0]:.3f}, hour = {model.

Key finding:
Rainfall has a very weak relationship with transport usage.
Time of day has a stronger influence, especially during commuting h
"""
    return context


def extract_user_features(user_query):
    query = user_query.lower()

    hour = None
    match = re.search(r'(\d{1,2})\s*(am|pm)', query)

    if match:
        hour = int(match.group(1))
        period = match.group(2)

        if period == "pm" and hour != 12:
            hour += 12
        elif period == "am" and hour == 12:
            hour = 0

    if hour is None:
        hour = int(data['hour'].mean())

    rainfall = data['rainfall'].mean() if "rain" in query or "wet"

    location = "Melbourne CBD" if "melbourne" in query or "cbd" in

    return hour, rainfall, location


def build_prompt(user_query, rag_context):
    prompt = f"""
You are an AI traffic explanation assistant.
```

```
User question:
{user_query}

Use only the retrieved context below to answer.

{rag_context}

Instructions:
1. Explain the likely traffic or transport usage condition.
2. Mention whether rainfall is important.
3. Mention whether time of day is important.
4. Give one practical recommendation.
5. Keep the answer clear and human-readable.
"""
    return prompt


def rag_traffic_assistant(user_query):
    rag_context = retrieve_rag_context(user_query)
    prompt = build_prompt(user_query, rag_context)

    hour, rainfall, location = extract_user_features(user_query)

    prediction_input = pd.DataFrame({
        "rainfall": [rainfall],
        "hour": [hour]
    })

    predicted_usage = model.predict(prediction_input)[0]
    avg_usage = data['count'].mean()

    usage_level = "higher than average" if predicted_usage >= avg_u

    if hour in [7, 8, 9, 16, 17, 18]:
        time_explanation = "The selected time is during a common co
    elif hour < 6:
        time_explanation = "The selected time is early morning, whe
    else:
        time_explanation = "The selected time is outside the strong

    if rainfall > 0:
        rainfall_explanation = "Rainfall was detected, but the retr
    else:
        rainfall_explanation = "No rainfall condition was detected,

    response = f"""
User question:
{user_query}

Retrieved RAG context used:
{rag_context}

Prompt engineered for the assistant:
{prompt}
```

```
Generated AI-style answer:
The predicted transport usage near {location} at {hour}:00 is {usag

{time_explanation}
{rainfall_explanation}

Based on the retrieved analysis, time of day is more important than

Recommendation:
Allow extra travel time during peak commuting hours, especially if
"""

    return response


user_question = input("Ask a traffic/weather question: ")
print(rag_traffic_assistant(user_question))
```

```
User question:
What is the best time to travel near Southbank if I want to avoid pe
ak transport activity?

Retrieved RAG context used:

Retrieved analytical context:

Average transport usage: 13.45
Average rainfall: 1.41 mm
Rainfall vs transport usage correlation: -0.007
Average usage on rainy days: 13.12
Average usage on non-rainy days: 13.63
Linear Regression R2 score: 0.009
Model coefficients: rainfall = -0.067, hour = 0.526

Key finding:
Rainfall has a very weak relationship with transport usage.
Time of day has a stronger influence, especially during commuting ho
urs.


Prompt engineered for the assistant:

You are an AI traffic explanation assistant.

User question:
What is the best time to travel near Southbank if I want to avoid pe
ak transport activity?

Use only the retrieved context below to answer.


Retrieved analytical context:

Average transport usage: 13.45
Average rainfall: 1.41 mm
Rainfall vs transport usage correlation: -0.007
```

```
Average usage on rainy days: 13.12
Average usage on non-rainy days: 13.63
Linear Regression R2 score: 0.009
Model coefficients: rainfall = -0.067, hour = 0.526

Key finding:
Rainfall has a very weak relationship with transport usage.
Time of day has a stronger influence, especially during commuting ho
urs.


Instructions:
1. Explain the likely traffic or transport usage condition.
2. Mention whether rainfall is important.
3. Mention whether time of day is important.
4. Give one practical recommendation.
5. Keep the answer clear and human-readable.


Generated AI-style answer:
The predicted transport usage near selected location at 12:00 is low
er than average.

The selected time is outside the strongest peak period, so usage may
be moderate.
No rainfall condition was detected, so weather is unlikely to strong
ly affect usage.

Based on the retrieved analysis, time of day is more important than
rainfall for explaining transport activity.

Recommendation:
Allow extra travel time during peak commuting hours, especially if t
he weather is wet.
```

Prompt Engineering and RAG Interpretation

This module demonstrates prompt engineering by using a structured prompt with clear instructions for the assistant. It also demonstrates RAG by retrieving relevant analytical context from the dataset and model results before generating the final explanation.

The user can type a natural-language question, and the system extracts useful information such as time and rainfall condition. The retrieved context is then combined with model predictions to produce a clear traffic explanation and recommendation.